

HASHING

PROBLEM 10

QUESTION:

DS	Session	Hashing	Question Information	Level 1	Challenge 100
Problem Description Mathison has uncovered an ancient piece of paper on which are written N integers. This set of numbers will be referred to as A. Mathison learned in History class that a group of integers is unique if and only if their total is divisible by a mythological constant M. Mathison tries to find out how many distinct triplets of numbers, from the piece of paper, have their sum divisible by M. Unfortunately, this problem is quite hard to crack and he needs your help. Constraints $1 \leq N \leq 2 \times 10^5$ $1 \leq M \leq 10^4$ $0 \leq A[i] \leq 2 \times 10^9$ Input The first line of the input file contains two space-separated integers, N and M, representing the number of integers and the mythical constant. The next line contains N space-separated integers, where the i^{th} integer represents $A[i]$. Output The output file should contain only one integer, the answer to Mathison's question. Explanation for Test case 1 There are 26 special trios: {0, 1, 2}; {0, 1, 8}; {0, 2, 5}; {0, 2, 6}; {0, 2, 9}; {0, 3, 7}; {0, 5, 8}; {0, 6, 8}; {0, 8, 9}; {1, 2, 7}; {1, 3, 4}; {1, 5, 6}; {1, 5, 9}; {1, 6, 9}; {1, 7, 8}; {2, 4, 8}; {2, 5, 7}; {2, 6, 7}; {2, 7, 9}; {3, 4, 5}; {3, 4, 6}; {3, 4, 9}; {5, 6, 9}; {5, 7, 8}; {6, 7, 8}; {7, 8, 9}. Note: Here we only show the positions (0-indexed) in each trio.					
Logical Test Cases Test Case 1 INPUT (STDIN) 10 5 1 10 4 3 2 5 0 1 9 5 EXPECTED OUTPUT 26 Test Case 2 INPUT (STDIN) 10 5 11 10 14 31 21 15 10 11 9 51 EXPECTED OUTPUT 31					
Mandatory Test Cases Test Case 1 KEYWORD while(i<k)					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 10 Test Case 2 TOKEN COUNT 450 Test Case 3 NLOC 55					

SOURCE CODE:

```
#include <stdio.h>

long long remainder_freq[10005];

int main() {
    int total_nums, divisor_m;
    if (scanf("%d %d", &total_nums, &divisor_m) != 2) return 0;

    for (int i = 0; i < total_nums; i++) {
        long long val;
        scanf("%lld", &val);
        int rem = ((val % divisor_m) + divisor_m) % divisor_m;
        remainder_freq[rem]++;
    }

    long long total_triplets = 0;

    for (int i = 0; i < divisor_m; i++) {
        if ((3LL * i) % divisor_m == 0 && remainder_freq[i] >= 3) {
            total_triplets += remainder_freq[i] * (remainder_freq[i] - 1) * (remainder_freq[i] - 2) / 6;
        }
    }

    for (int i = 0; i < divisor_m; i++) {
        int rem_k = (divisor_m - (2 * i) % divisor_m) % divisor_m;
        if (rem_k != i && remainder_freq[i] >= 2 && remainder_freq[rem_k] >= 1) {
            total_triplets += (remainder_freq[i] * (remainder_freq[i] - 1) / 2) * remainder_freq[rem_k];
        }
    }

    for (int i = 0; i < divisor_m; i++) {
        for (int j = i + 1; j < divisor_m; j++) {
            int k = (divisor_m - (i + j) % divisor_m) % divisor_m;
            if (j < k) {
                total_triplets += remainder_freq[i] * remainder_freq[j] * remainder_freq[k];
            }
        }
    }

    printf("%lld\n", total_triplets);
    return 0;
}
```

OUTPUT:

```
10 5
11 18 14 31 21 15 10 11 9 51
26
```

```
10 5
11 10 14 31 21 15 10 11 9 51
31
```